

DevOps Engineer Exam

Objective

Test your skills with **Docker**, **Git**, and **Terraform** by self-hosting a Git server and managing infrastructure as code.

 **Note:** Submit within **3 days** of receiving these instructions.

Prerequisites

- Docker & Docker Compose installed.
- Terraform installed.
- Git installed.

Instructions

Part 1: Git Server Setup

1. Create a `docker-compose.yml` file to deploy Gitea.
2. Use the `gitea/gitea:1.22.6` image.
3. Expose port `3000` (web) to the host.
4. Persist data using a volume.
5. Start the service and access Gitea at `http://localhost:3000`.
6. Complete the initial setup and register a new user to create your credentials.

Part 2: Infrastructure as Code

 **Note: Terraform Project Structure Requirements:** Your Terraform code must be organized into the following files:

- `main.tf` - Main resource definitions and provider configurations
- `variables.tf` - All variable declarations with descriptions and types
- `outputs.tf` - Output definitions for important values
- **No hardcoded values** in `main.tf` — use variables.
- **Use a reusable module:** the Docker resources (Part 3) must live in a module under `modules/docker-app/` (with its own `main.tf`, `variables.tf`, `outputs.tf`) and be consumed from the root `main.tf` via a `module` block.

- **Add input validation:** the Nginx host-port variable must include a `validation` block that rejects ports outside the `1024–65535` range.

1. **Generate an Access Token** in Gitea:

- Go to **Settings -> Applications -> Manage Access Tokens**.
- Generate a new token with the following scopes: `write:repository` , `write:user` , `read:organization` , `write:issue` .
- Save this token securely (you'll need it for Terraform and API calls).

2. Create Terraform configuration files (`main.tf` , `variables.tf` , `outputs.tf`) in the root (or a dedicated `terraform` directory).

3. Configure the **Gitea Provider** in your Terraform code using variables.

4. Write Terraform code to **create a new private repository** named `terraform-docker-exam` using the `gitea_repository` resource. The repository must be set to **private** (not public).

5. **Configure Branch Protection** on the repository using the `gitea_repository_branch_protection` resource:

- Protect the `main` branch.
- Enable at least one protection rule (e.g., prevent force push, require reviews, etc.).

6. Define **outputs** for:

- Repository clone URL (HTTP)
- Repository web URL

7. Initialize a git repository in your project folder (if not already done).

8. Apply your Terraform configuration. This should create the repository on your local Gitea server.

9. Add the Gitea remote pointing to your new repository (using the clone URL from Terraform output).

Part 3: Docker via Terraform

 **Note: Container Labeling Requirements:** All Docker containers must include the following labels:

- `project` = "devops-exam"
- `environment` = "development"
- `managed-by` = "terraform"

1. Extend your Terraform configuration to also use the **Docker Provider**.

2. **Create a custom Docker network** named `exam-network` using the `docker_network` resource.

3. **Create a Docker volume** named `exam-web-data` using the `docker_volume` resource (it will be mounted into Nginx below).

4. Write Terraform code to:

- Pull the `nginx:1.27.4` image.
- Start an Nginx container named `exam-web-server` .
- Map port `8080` on the host to port `80` on the container.
- Attach the container to `exam-network` .

- Mount the `exam-web-data` volume into the container (e.g. at `/var/cache/nginx`).
- Add a native container `healthcheck` block so Docker reports the container as `healthy`.
- Apply the required labels.

5. Create a health-check container:

- Pull the `curlimages/curl:8.17.0` image.
- Start a container named `exam-health-checker`.
- Attach the container to `exam-network`.
- Configure it to run: `sh -c "while true; do curl -sf http://exam-web-server:80 || echo 'Health check failed'; sleep 30; done"`
- This container must **depend on** the Nginx container.
- Apply the required labels.

6. Define **outputs** for:

- Nginx container ID
- Health checker container ID
- Network ID

7. Run `terraform apply` to create the Docker resources.

8. Verify you can access Nginx at `http://localhost:8080`.

9. Verify the health-checker container is running and checking Nginx.

Part 4: Specific Git Workflow

 **Note:** All commits must follow [Conventional Commits](#).

1. Create a `.gitignore` file to exclude appropriate files for a Terraform project.
2. Commit your initial Terraform code to the `main` branch using conventional commit messages.
3. Push the code to your **Gitea** repository.
4. Create and switch to a new branch named `update-port`.
5. Change the Nginx host port mapping to `8081` in your Terraform code.
6. Apply the change using Terraform.
7. Commit the change using a conventional commit message and push the `update-port` branch to Gitea.

Submission

 **Note:** Follow the instructions carefully. Your submission will be validated using an automated script that checks for exact file names, resource names, and configurations as specified.

1. Create an `outputs` directory and capture all required outputs:

```
mkdir -p outputs
terraform show > outputs/terraform-state.txt
terraform output > outputs/terraform-output.txt
terraform validate -json > outputs/terraform-validate.json
terraform plan -no-color > outputs/terraform-plan.txt # must show "No changes"
docker ps > outputs/docker-ps.txt
docker network inspect exam-network > outputs/docker-network.txt
docker logs exam-health-checker > outputs/health-checker-logs.txt 2>&1
docker inspect --format '{{.State.Health.Status}}' exam-web-server > outputs/nginx-health.txt
curl -s http://localhost:8081 > outputs/nginx-welcome.html
```

2. Capture Gitea API outputs (replace `USER` and `TOKEN`):

```
REPO="http://localhost:3000/api/v1/repos/USER/terraform-docker-exam"
curl -s -H "Authorization: token TOKEN" $REPO > \
    outputs/gitea-repo.json
curl -s -H "Authorization: token TOKEN" $REPO/branch_protections > \
    outputs/gitea-branch-protection.json
```

3. Push your final code (Terraform files, `docker-compose.yml`, `.gitignore`, and `outputs/` directory) to a **public GitHub repository**.

4. Send an email using the template below.

Email Template

To: `devops-exam@easyequities.com.ph`

Subject: `DevOps Exam Submission - [Your Name]`

Body:

Hi,

Please find my DevOps Engineer Exam submission at the following GitHub repository:

`https://github.com/[your-username]/[your-repo-name]`

Thank you.

[Your Name]

Good luck!